

1. Python is a high level programming language used for web development, data science, AIML, automation and more.

- * Interpreted language (no need to compile).

- * Simple English-like syntax.

- * Cross platform (platform independent) (windows, linux, mac)

- * Open source (free of cost).

Features of Python programming language:-

1. Simple and easy to learn:- Python has a clear and readable syntax, similar to english, which makes it beginner-friendly.
2. Interpreted Language:- Python executes code line by line, so there is no need for compilation. This makes debugging easier.
3. High level language:- It hides complex details of memory management, allowing programmers to focus on problem solving.
4. Dynamically Typed:- You don't need to declare variable types explicitly. The type is decided at runtime.
5. Platform Independent (Portable):- Python programs can run on different operating systems like windows, linux, and macos without changes.
6. Object-oriented:- Supports concepts like classes, objects, inheritance and polymorphism.
7. Large standard library:- Python is freely available and has a large community support.
8. Open Source and Free:- Python is freely available and has a large community support.
9. Extensive and embeddable:- Python can be integrated with other languages like C and C++.
10. Supports Multiple programming paradigms:- It supports procedural, object-oriented and functional programming styles.

Applications of Python:-

1. Web development:- Used to create webseries and web apps.
2. Data Analysis and Data Science:- Helps in processing, analyzing and visualizing data.
3. Artificial Intelligence & Machine learning:- Used to build smart systems like chatbots and recommendation systems.
4. Automation (Scripting):- Automates repetitive tasks such as file handling and testing.
5. Software Development:- Used to develop desktop applications and system software.
6. Web Scrapping:- Extracts data from websites.
7. Scientific computing:- Used in research, engineering and mathematical calculations.
8. Game Development:- Used to create simple games and prototypes.
9. Cyber Security:- Helps in ethical hacking and security testing.
10. Internet of Things (IOT):- Used in devices like Raspberry Pi for hardware control.

Different types of errors:-

There are three types of errors.

1. Syntax error
2. Runtime error
3. Logical error.

1. Syntax error:-

=> Missing colons (:) after if/else/for statements.

=> Mismatched parentheses or quotes.

=> Incorrect indentation.

=> Misspelled keywords.

Ex:- if x > 5

print(x) # syntax error: missing colon

Fix:- if x > 5:

2. Runtime error:- Runtime errors happen while the code's executing

Examples:- Division by zero.

Out-of-range values.

Using undefined variables.

Ex:- x = 5/0 # zero division error.

3. Logical error:- Code runs fine, but doesn't give the right output - that's a logical error! Happens when the logic's flawed.

Ex:- def add(x, y):

return x-y # oops!

To Fix:- return x+y.

* Modes of python:-

=> Python is operated by two types.

1. Interactive mode
2. Script mode / Normal mode.

1. Interactive mode:-

In this mode, lines of code are directly written and executed in the python interpreter shell, which instantaneously provides the results.

2. Normal / Script Modes:-

In this mode, the source code is saved into a file with python extension and the files executed by the python interpreter.

Interpreter

- * An interpreter translates & executes the program line by line.
- * Interpreted programs run slower, because translation happens during execution.
- * An interpreter shows errors one by one and stops at the 1st error.
- * An interpreter does not generate a separate executable file.
- * Ex :- python, javascript.

Compiler.

- * A compiler translates the entire program at once into machine code.
- * Compiled programs run faster, since they are already converted into machine code.
- * A compiler shows all errors together after compilation.
- * A compiler generates an executable file.
- * Ex :- C, C++

*Print Function:-

The `print()` function is used in python primarily, to display output to the console, allowing humans to see the results of a program's execution.

Syntax:- `print(object(s), sep=' ', end='\n', file=sys.stdout, flush=False)`.

Example:- `print("Hello", "World", sep="-")` → Hello-World.

⇒ We can use / take ^{many} number of objects.

⇒ Separate objects using "sep".

⇒ By default each print statement ends with `\n`. We can change it using "end".

⇒ By default output is printed to output screen using "file".

⇒ By default data is stored in output buffer before sending to the output screen we can change this "flush".

* Explanation of Assignment statements.

=> Assignment statements/operators are used to assign the values.

=> An assignment statement is a fundamental programming construct that stores a value in a variable, replacing any previous values.

=> It uses an operator (typically =) to evaluate an expression.

=> Ex:-

- * $x = 10$ (Assigns 10 to variable x)
- * $y = x + 5$ (evaluates $10 + 5$, then assigns 15 to y)
- * $counter++$ (equivalent to $counter = counter + 1$)

* Explanation of Selective statements.

=> ~~S~~ In Python, "selective statements", also known as conditional statements or decision-making statements.

=> Selective statements are used to execute specific blocks of code based on whether certain conditions are true or false.

=> The primary selection statements are if, elif, and else.

=> If statement: - This is the simplest form. The code block within the if statement runs only if the condition is true.

Ex: age = 20

if age >= 18:

print("You are an adult") # This line is indented and will execute.

print("You can vote") # This line is also indented and will execute.

=> If-else statement: - This provides two possibilities. One block of code is executed if the condition is True, and a different block is executed if the condition is false.

Ex: age = 16

if age >= 18:

print("You can vote")

else:

print("You can't vote") # This block will execute.

=> If-elif-else statement:- Also known as an if ladder or chained conditional, this allows you to check multiple conditions sequentially.

The code block for the first condition that evaluates to True is executed, and the rest are skipped.

Ex:- score = 75

```
if score >= 90:  
    print("Grade A")
```

```
elif score >= 70:  
    print("Grade B") # This block will execute
```

```
else:  
    print("Grade C")
```